BOX-QQL Reference Manual

Binary **O**ptimized e**X**ecution **Q**uery and **Q**ualification **L**anguage

1. Introduction & Language Overview

BOX-QQL is the high-performance native database query and manipulation language powering MiniDB. It seamlessly unifies custom minimalist syntax with standard SQL aliases to manage local schema files, execute relational joins, and run optimized hash index lookups.

2. Core Commands & SQL Aliases

BOX-QQL supports both native commands and common SQL alternatives interchangeably:

BOX-QQL Native	SQL Alias	Description
`GET`	`SELECT`	Retrieves columns and rows from one or more tables.
`ADD`	`INSERT`	Appends new record rows into a target table.
`CHANGE`	`UPDATE`	Modifies data values matching a condition filter.
`REMOVE`	`DELETE`	Deletes rows from a target table.
`BUILD`	`CREATE / DROP`	Manages database tables and structural schemas.
`INDEX`	`CREATE INDEX`	Builds in-memory hash indexes for O(1) lookups.

3. Data Definition & Schema Operations

BUILD TABLE / CREATE TABLE

Creates a new table with strict data type validation (`INT`, `TEXT`, `FLOAT`, `BOOL`).

```
BUILD TABLE employees WITH id INT name TEXT salary FLOAT active BOOL;
```

4. Data Manipulation & Retrieval Queries

GET / SELECT & Filtering

Supports wildcard selection, relational table joins, and conditional filtering using `FILTER` or `WHERE`.

```
GET name salary FROM employees FILTER salary > 60000;
```

5. Aggregate Functions

Analytical Calculations

Perform summary operations across datasets using `COUNT`, `SUM`, `AVG`, `MAX`, and `MIN`.

```
GET COUNT(*), SUM(salary) FROM employees FILTER active = True;
```

6. Logical Operators & Special Predicates

Complex Conditions

Build precise expressions using `AND`, `OR`, `NOT`, `IS NULL`, `LIKE`, `IN`, and `BETWEEN`.

```
GET * FROM employees FILTER age BETWEEN 25 AND 40 AND active IS True;
```

7. High-Performance Indexing

INDEX Operations

Optimize heavy query workloads by indexing specific table columns into high-speed memory maps.

```
INDEX employees ON id;
```